Binary CCV Versus Continuous-Treatment CCV

This document compares the proof idea behind the original binary-treatment CCV argument with the continuous-treatment extension. It is written to be readable without consulting the Lean formalization.

The main message is simple:

- In the binary case, treatment algebra collapses because `W_i^2 = W_i`.
- In the continuous case, that collapse is unavailable, so the proof keeps the full treatment-assignment covariance kernel.

The binary proof is therefore a treatment-share and cluster-sum proof. The continuous proof is a quadratic-form and covariance-kernel proof.

1. Common Setup

Think of a residualized regression after applying fixed effects or other controls. For observation or cell `i`, write:

```
Ytilde_i = beta * Dtilde_i + u_i.
```

Here:

- `Ytilde_i` is the residualized outcome;
- `Dtilde_i` is the residualized treatment;
- `beta` is the scalar treatment coefficient;
- `u_i` is the residualized structural error or potential-outcome residual.

The scalar FWL/OLS coefficient is:

```
betaHat = (sum_i Dtilde_i Ytilde_i) / (sum_i Dtilde_i^2).
```

Let:

```
S = sum_i Dtilde_i^2.
```

If `S != 0`, substituting `Ytilde_i = beta * Dtilde_i + u_i` gives the finite sample score expansion:

```
betaHat - beta
  = (sum_i Dtilde_i u_i) / S.
```

This identity is common to both the binary and continuous cases. The difference between the two proofs is how they handle the variance of the score:

```
sum_i Dtilde_i u_i.
```

2. Original Binary-Treatment Proof

2.1 Binary Treatment Algebra

In the binary case, treatment is an indicator:

```
W_i in {0, 1}.
```

The decisive algebraic fact is:

```
W_i^2 = W_i.
```

This means that second moments of treatment can often be rewritten as first moments. If `Wbar` is the average treatment share in a finite group,

```
Wbar = (1 / n) * sum_i W_i,
```

then the within-group residualized treatment second moment is:

```
(1 / n) * sum_i (W_i - Wbar)^2 = Wbar * (1 - Wbar).
```

The calculation is:

```
sum_i (W_i - Wbar)^2
  = sum_i (W_i^2 - 2 W_i Wbar + Wbar^2)
  = sum_i W_i - 2 Wbar sum_i W_i + n Wbar^2
  = n Wbar - 2 n Wbar^2 + n Wbar^2
  = n Wbar (1 - Wbar).
```

Dividing by `n` gives:

```
mean((W_i - Wbar)^2) = Wbar * (1 - Wbar).
```

This identity is the key simplification in the original binary proof. The within-group treatment variation can be described by the scalar treatment share `Wbar`, not by a full covariance matrix.

2.2 Clustered Score Algebra

Now suppose observations are grouped into clusters `g`. Let `psi_gi` be the score contribution for observation `i` inside cluster `g`.

The cluster score is:

```
Psi_g = sum_i psi_gi.
```

The clustered variance estimator uses squared cluster scores:

```
V_cluster = S^(-2) * sum_g Psi_g^2
          = S^(-2) * sum_g (sum_i psi_gi)^2.
```

The square of the cluster sum expands as:

```
(sum_i psi_gi)^2
  = sum_i sum_j psi_gi psi_gj
  = sum_i psi_gi^2 + sum_{i != j} psi_gi psi_gj.
```

Therefore:

```
V_cluster
  = S^(-2) * sum_g sum_i psi_gi^2
    + S^(-2) * sum_g sum_{i != j} psi_gi psi_gj.
```

The first term is the Eicker-Huber-White or robust diagonal part. The second term is the within-cluster cross-product correction. The clustered variance estimator is exactly what you get when you allow arbitrary within-cluster score correlation and no across-cluster score correlation.

2.3 Why the Binary Proof Is Short

The original binary proof is short because the binary restriction imposes a strong moment identity:

```
W_i^2 = W_i.
```

That identity gives:

```
within-group residualized treatment variance = Wbar * (1 - Wbar).
```

Then the rest is cluster-sum algebra:

```
cluster variance = robust diagonal part + within-cluster cross-products.
```

So the proof path is:

```
binary treatment
  -> W_i^2 = W_i
  -> residualized treatment variance depends on Wbar
  -> clustered score square expands into pairwise products
  -> clustered variance estimator matches the design variance
```

3. Continuous-Treatment Extension

3.1 What Fails Outside the Binary Case

For a continuous treatment `D_i`, the binary identity is false:

```
D_i^2 != D_i
```

in general. Therefore the within-group residualized treatment variance cannot be reduced to a treatment share. The expression:

```
mean((D_i - Dbar)^2)
```

depends on the full second moment of `D`, not only on its mean.

More importantly, when treatment assignment is random or design-based, the variance of the score:

```
sum_i D_i u_i
```

depends on all pairwise assignment covariances:

```
Cov(D_i, D_j).
```

The continuous proof must therefore keep the covariance structure explicit.

3.2 Covariance Kernel

Let the treatment assignment vary across design states `omega`. For each state, there is a treatment vector:

```
D(omega) = (D_1(omega), ..., D_N(omega)).
```

Let `p(omega)` be the probability of assignment state `omega`. Define the design expectation:

```
E_p[X] = sum_omega p(omega) X(omega).
```

The centered treatment coordinate is:

```
Dcenter_i(omega)
  = D_i(omega) - E_p[D_i].
```

The treatment assignment covariance kernel is:

```
Sigma_ij
  = E_p[Dcenter_i Dcenter_j]
  = sum_omega p(omega)
      (D_i(omega) - E_p[D_i])
      (D_j(omega) - E_p[D_j]).
```

This `Sigma` replaces the binary treatment-share formula.

3.3 Score Variance as a Quadratic Form

Consider the linear score:

```
L(omega) = sum_i D_i(omega) u_i.
```

Its design variance is:

```
Var_p(L)
  = E_p[(L - E_p[L])^2].
```

Because:

```
L(omega) - E_p[L]
  = sum_i (D_i(omega) - E_p[D_i]) u_i
  = sum_i Dcenter_i(omega) u_i,
```

we get:

```
Var_p(L)
  = E_p[(sum_i Dcenter_i u_i)^2]
  = E_p[sum_i sum_j Dcenter_i Dcenter_j u_i u_j]
  = sum_i sum_j u_i E_p[Dcenter_i Dcenter_j] u_j
  = sum_i sum_j u_i Sigma_ij u_j.
```

In matrix notation:

```
Var_p(L) = u' Sigma u.
```

This is the central continuous-treatment replacement for the binary `Wbar(1 - Wbar)` identity.

3.4 Sandwich Variance

The FWL score expansion is:

```
betaHat - beta = S^(-1) * L,
```

where:

```
S = sum_i Dtilde_i^2.
```

Therefore:

```
Var_p(betaHat - beta)
  = Var_p(S^(-1) L)
  = S^(-2) Var_p(L)
  = S^(-2) u' Sigma u.
```

This is the continuous-treatment design covariance variance:

```
V_design = S^(-2) * u' Sigma u.
```

The feasible estimator is:

```
dcCCV = S^(-2) * uhat' SigmaHat uhat.
```

If:

```
uhat = u
SigmaHat = Sigma,
```

then:

```
dcCCV = V_design.
```

So in the exact finite-sample continuous case, the proof is also algebraic. It just uses a full covariance kernel instead of a binary treatment-share identity.

4. Fixed Denominator Versus Random Denominator

The cleanest continuous case assumes the OLS denominator is fixed across assignment states:

```
sum_i D_i(omega)^2 = S
```

for every `omega`.

Then:

```
betaHat(omega) - beta
  = S^(-1) * sum_i D_i(omega) u_i.
```

The covariance kernel should be computed from raw residualized treatment assignment `D`.

If the denominator is random, the exact proof still works, but the score regressor changes. For each assignment state, define:

```
Dnorm_i(omega)
  = D_i(omega) / (sum_k D_k(omega)^2).
```

Then:

```
betaHat(omega) - beta
  = sum_i Dnorm_i(omega) u_i.
```

The relevant covariance kernel is now:

```
Cov(Dnorm_i, Dnorm_j),
```

not:

```
Cov(D_i, D_j).
```

This distinction is minor in the fixed-denominator benchmark but important in general continuous-treatment designs.

5. How the Two Proofs Compare

Binary Proof

The binary proof uses a reduction:

```
W_i in {0,1}
  -> W_i^2 = W_i
  -> mean((W_i - Wbar)^2) = Wbar(1 - Wbar)
  -> cluster variance is expanded by pairwise score products
```

It relies on the treatment being an indicator. The treatment share `Wbar` carries the relevant second-moment information.

Continuous Proof

The continuous proof keeps the second moments:

```
D_i real-valued
  -> no W_i^2 = W_i simplification
  -> keep Sigma_ij = Cov(D_i, D_j)
  -> Var(sum_i D_i u_i) = u' Sigma u
  -> Var(betaHat - beta) = S^(-2) u' Sigma u
```

It works for binary treatment too, because binary treatment has a covariance kernel. But for binary treatment, the kernel can often be simplified using `W_i^2 = W_i`. The continuous proof deliberately does not use that shortcut.

6. Exact Finite-Sample Case

The simplest continuous-treatment theorem requires:

1. A finite assignment probability model `p(omega)`.
2. A residualized treatment vector `D(omega)`.
3. A fixed residual vector `u`.
4. A nonzero OLS denominator.
5. A covariance kernel:

   ```
   Sigma_ij = Cov_p(D_i, D_j)
   ```

   or, with random denominator,

   ```
   Sigma_ij = Cov_p(Dnorm_i, Dnorm_j).
   ```

6. Exact feasible inputs:

   ```
   uhat = u
   SigmaHat = Sigma
   ```

Then:

```
dcCCV = true finite-design variance of betaHat - beta.
```

In the fixed-denominator case:

```
dcCCV
  = S^(-2) * u' Sigma u
  = Var_p(betaHat - beta).
```

In the random-denominator case:

```
dcCCV
  = u' Cov_p(Dnorm) u
  = Var_p(betaHat - beta).
```

The continuous proof is therefore not weaker than the binary proof. It is more general, but it asks for the covariance object explicitly.

7. Feasible Asymptotic Case

In empirical applications, the true covariance kernel `Sigma` is usually not known. The feasible estimator uses:

```
dcCCV_n = S_n^(-2) * uhat_n' SigmaHat_n uhat_n.
```

The asymptotic proof needs conditions ensuring:

```
dcCCV_n / V_design -> 1
```

or equivalently:

```
designSE_n / dcSE_n -> 1.
```

A minimal high-level assumption set is:

1. Score CLT:

   ```
   linearized score_n -> Normal(0, 1) in distribution.
   ```

2. Asymptotic linearization:

   ```
   (betaHat_n - beta) / designSE_n
     = linearized score_n + remainder_n,
   ```

   with:

```
remainder_n -> 0 in probability.
```

3. Residual consistency:

```
uhat_n -> u.
```

4. Kernel consistency:

```
SigmaHat_n -> Sigma.
```

5. Regularity of the standard error:

```
V_design > 0,
dcSE_n != 0 eventually,
and the feasible SE ratio is measurable.
```

Under these assumptions:

```
(betaHat_n - beta) / dcSE_n -> Normal(0, 1).
```

Then confidence interval coverage follows from the usual continuity condition on the limiting normal distribution at the interval boundary.

8. How This Fits an Administrative Policy Design

For an administrative policy design, assignment may be governed by rules that depend on variables such as prior wages, eligibility thresholds, geography, industry, or employer characteristics. If all rule inputs and assignment probabilities were known, one could in principle compute `Sigma` directly.

If some rule inputs are only partially observed, exact `Sigma` is usually not available. Then the proof should use the feasible asymptotic route:

```
estimate Sigma by SigmaHat
prove SigmaHat -> Sigma
use dcCCV_n = S_n^(-2) uhat_n' SigmaHat_n uhat_n
```

The empirical design must justify kernel consistency. Common routes are:

- an iid or independent-cluster sample-kernel law of large numbers;
- a weak-dependence law of large numbers across administrative cells;
- a correctly specified model for the latent assignment inputs;
- repeated comparable assignment realizations from the same policy mechanism.

The proof does not require binary treatment. It requires enough structure to learn or know the covariance kernel governing continuous assignment variation.

9. Summary Table

Feature	Binary proof	Continuous proof
Treatment	`W_i in {0,1}`	`D_i in R`
Key moment identity	`W_i^2 = W_i`	no binary simplification
Residualized treatment variance	`Wbar(1 - Wbar)`	full covariance kernel
Main variance object	cluster score sums	`u' Sigma u`
Finite-sample exactness	from binary algebra and cluster expansion	from covariance-kernel quadratic form
Feasible empirical version	cluster-robust estimator	`S^(-2) uhat' SigmaHat uhat`
Main extra burden	cluster assignment assumptions	consistency of `SigmaHat`

10. Bottom Line

The original binary proof works because binary treatment collapses second moments into first moments. The continuous extension works by refusing to make that collapse: it keeps every second moment through the covariance kernel `Sigma`.

For the simplest exact continuous case, the proof is still finite-sample and algebraic:

```
FWL score expansion
  + finite design covariance identity
  + exact uhat and SigmaHat
  -> exact dcCCV design variance.
```

For the realistic feasible case, the proof becomes asymptotic:

```
score CLT
  + estimator linearization
  + residual consistency
  + SigmaHat consistency
  + nondegenerate standard errors
  -> valid studentized dcCCV inference.
```